

Agentic Coding

Beyond the Chatbox

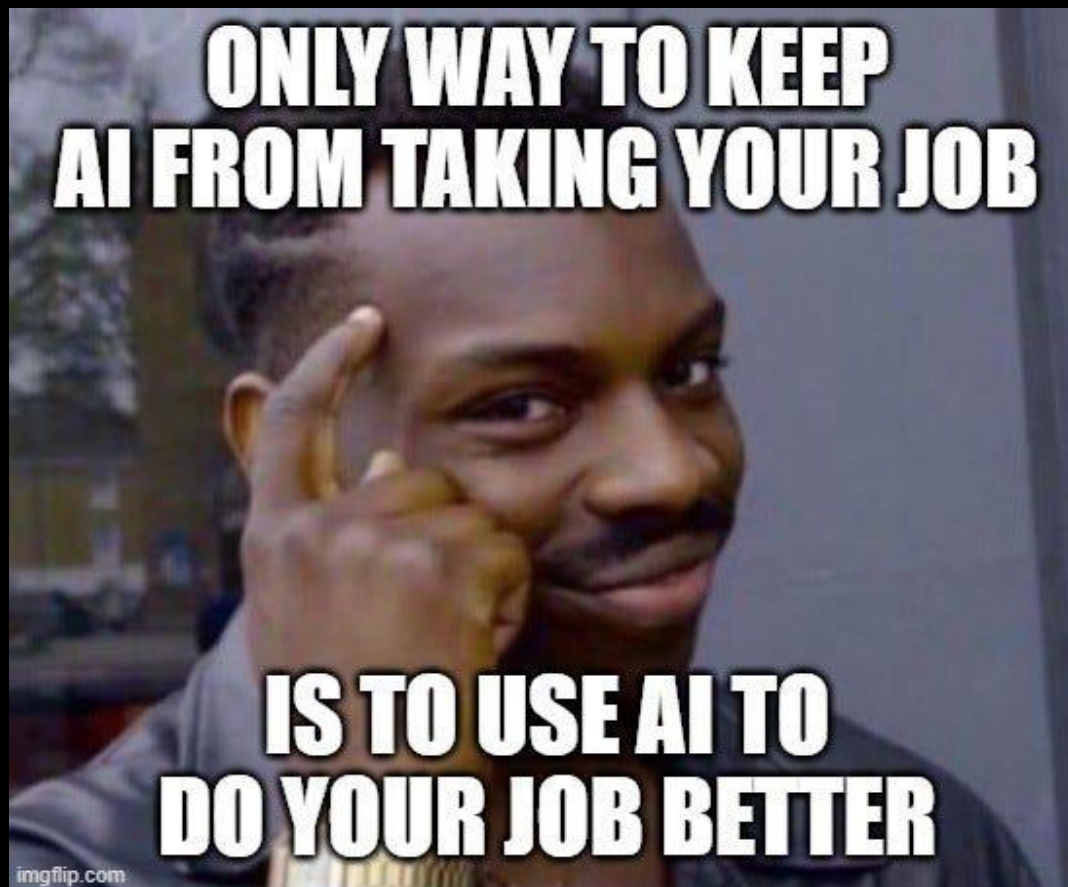
King Abdullah University of Science and Technology (KAUST)
KAUST Summer

Prepared by: Almoayyad Abuljdail

Table of Contents

1. The loop, not the prompt
2. Guardrails: permissions
3. Context, plans, and verification
4. MCP: tools, scoped
5. Skills, plugins, and hooks

**ONLY WAY TO KEEP
AI FROM TAKING YOUR JOB**



**IS TO USE AI TO
DO YOUR JOB BETTER**

Two Ways to Use It

- Bad use: prompt, then trust.
- Good use: context → plan → permission → act → verify → diff.

Note: This talk mainly uses Claude Code and Codex. If you don't have access to them, try OpenCode, an open-source alternative with a similar agentic workflow. I'd actually recommend trying it first before committing to paid frontier options.

**The Loop,
Not the
Prompt**

The Agent Loop

- An agent acts, observes the result, and decides the next action, until a goal is met or you stop it.
 - **Read:** grep the repo, open the files, load the memory files.
 - **Plan:** state the change and the files it will touch.
 - **Act:** edit files, run commands, install dependencies.
 - **Verify:** run the tests, open the page, read the diff.
- Step four is the one a chat window cannot do.
- Failure at step four feeds back into step one. That feedback is the product.

What the Evidence Says

- Agents are not automatically faster.
 - [METR, 2025](#): 19% slower, while estimating they were 20% faster.
 - [GitClear, 2026](#): duplicated code up 81%, refactoring down 70%.
 - [Terminal-Bench, 2026](#): frontier agents score under 65%.
 - [DORA, 2025](#): adoption correlates with lower delivery stability.
- AI amplifies the practice you already have.

Guardrails: Permissions

Permissions

- Autonomy you did not authorise is not speed.
 - [Plan, or read-only](#): it investigates and proposes. It cannot write.
 - [Auto-edit](#): writes files freely, asks before running commands.
 - [Full access](#): runs anything. Container or throwaway repo only.
- Nine documented data-destruction incidents by coding agents in fourteen months.
 - [Claude Code issue #10077](#): the permission system was on. It did not catch that the approved command would expand to the home directory.
- Commit before you prompt. Git does not prevent damage, it makes damage reversible.
- One branch per agent task. Read the diff before you accept it.
- Grant the least power that still finishes the task.

Context, Plans, and Verification

The Context Stack

- Separate what, why, how, and who.
 - `product.md`: what are we building? Users, scope, done-criteria.
 - `CLAUDE.md`: project memory for Claude Code. Conventions and commands.
 - `AGENTS.md`: the vendor-neutral equivalent. Codex reads this one.
 - `.claude/`: settings, skills, subagents, hooks, MCP configuration.
- Checked into the repo, so the team and every future session inherit it.
- Files make your standards portable across sessions and tools.

Writing a CLAUDE.md That Works

```
# Commands
- test:  pytest -q
- lint:  ruff check .

# Conventions
- Python 3.12, type hints required
- No new dependencies without asking

# Never
- Never touch migrations/
- Never commit or push unless asked
```

- Rules the agent can obey, not a description of the project.
- Keep it under 200 lines. Give it an owner. Review changes like code.
- Positive rules guide; negative rules stop the default behaviour.
- Every incident becomes a new “Never” line.
- Procedural steps belong in a skill, not in here.

Plan Before You Let It Type

- The cheapest place to catch a wrong approach is before the edit.
- Shift+Tab enters plan mode: it reads and proposes, and cannot write.
- Name the files. Do not make it guess.
- Say what “done” means before it starts.
- Ask for the root cause, not a patch. Reject the first plan if it is vague.
- Long task? Have it write plan.md, then execute that.
- Fresh session per task. Small scope beats a clever prompt.

Verification

- Never accept a claim the agent did not test.
 - **Tests:** give it the exact command, make it run it, and read the output.
 - **Browser:** Chrome or Playwright MCP, so it screenshots the page and looks at it.
 - **The diff:** git diff is the deliverable. Read it before merging.
- “Done” from an agent means “I stopped”, not “it works”.
- If it cannot be verified, it cannot be delegated.

Try This Yourself

- Pick one real bug in a repo you know. Commit first, then work on a branch.
 - **context**: point it at the files. State what “done” means before it starts.
 - **plan**: read-only. Ask for the root cause and the fix it proposes. Reject it if vague.
 - **permission**: allow edits. Do not allow commands yet.
 - **act**: let it edit. Watch which files it touches.
 - **verify**: make it run the test and show the output, not a summary of the output.
 - **diff**: git diff. Accept, or send it back with one sentence.
- About eight minutes. If it goes wrong, the diff you did not accept is the lesson.

Model or Effort?

- Two settings that both look like “make it better”. They are not the same.
 - **Model**: which frozen weights answer you. Capability, and price per token.
 - **Effort**: how long it thinks, how many files it reads, how much it verifies.
- Higher effort is not better. The default is calibrated to what most people would want spent on the task.
 - **Too high**: it over-verifies, and overthinking degrades effectiveness.
 - **Too low**: it asks you for context instead of working it out.
- Wrong answer, good context? Did it not know enough, or not try hard enough?
- Before you touch either dial, fix the context you gave it.

<https://arena.ai/leaderboard/agent> This website could help you choose the best model for a specific task

How to Decide

- Start at default effort, on the middle model. Tune nothing until something fails.
- When it fails, change the context first, not a setting.
 - Vague prompt, missing files, wrong tools connected, no skill for the procedure.
- Only then classify: did it not know enough, or did it not try hard enough?
- Ratchet back down when the work goes routine. Most people escalate once and never return.
- Larger models absorb ambiguity. Smaller ones need precise instructions.
- No benchmark answers this for your codebase. Try the cheaper option, keep /usage open.

**MCP:
Tools,
Scoped**

MCP, Used Correctly

- One open protocol for giving an agent tools, resources, and prompts, instead of one bespoke integration per service.
- Adding a server is easy. Scoping it is the skill.
 - **local**: just you, just this repo. The default, and right for experiments.
 - **project**: lands in .mcp.json, commits with the code, the team inherits it.
 - **user**: every repo you open. Reserve it for things you always want.

```
claude mcp add playwright -- npx @playwright/mcp@latest  
claude mcp add --transport http linear https://mcp.linear.app/mcp  
codex mcp add playwright -- npx ...
```

MCP Hygiene

- Every server you connect is a new trust boundary.
 - **Context cost:** every tool definition is spent on every turn.
 - **Tool poisoning:** malicious text in tool metadata, fired every session.
 - **Credentials:** an unvetted npx package runs as you.
- Enable per project. Prefer first-party servers. Read-only where possible.
- Treat fetched content as data, never as instructions.

Skills, Plugins, and Hooks

Skills

- A folder of instructions the agent loads only when it is relevant.
- Write one when you keep pasting the same procedure.
 - **Cheap to keep:** only the description sits in context until it triggers.
 - **Loaded on demand:** the body arrives when needed.
- A skill is tribal knowledge the agent can execute.

How to Make One

```
.claude/skills/deploy-api/    <- directory name becomes /deploy-api
  SKILL.md                   <- the only required file. Keep it small.
  checklist.md               <- read only when SKILL.md points at it
  rollback.sh                <- Claude can run it

---
description: Deploy the API to staging or prod. Use when asked
  to ship, release, or roll back the API.
disable-model-invocation: true
allowed-tools: Bash(${CLAUDE_SKILL_DIR}/rollback.sh *) Read
---
```

- Only the description really matters. Write what it does and when to use it. That is the trigger.
- Keep SKILL.md small, because it stays in context once loaded. Put large references in supporting files, read only when needed.
 - **disable-model-invocation**: only you can run it, never Claude.
 - **user-invocable: false**: only Claude runs it, hidden from the / menu.
 - **paths**: auto-activates only when working with matching files.
 - **allowed-tools**: pre-approves tools. disallowed-tools is what removes them.

THE ESSENTIALS

- **Superpowers** (obra): brainstorming, systematic debugging, TDD enforcement, and execution planning, as slash commands.
 - `/superpowers:execute-plan` runs a written plan end to end.

github.com/obra/superpowers 272k stars
- **Skill Seekers** (yusufkaraaslan): point it at a docs site, a PDF, or a repo, and it generates a skill.
 - Quick test: point it at docs the model has never seen, then ask the same question before and after.

github.com/yusufkaraaslan/Skill_Seekers 14.8k stars
- **find-skills** (vercel-labs/skills): the most installed skill of all. Its whole job is finding you the others.
 - `npx skills find <query>` searches skills.sh. Browse it yourself if you would rather look by hand.

skills.sh · github.com/vercel-labs/skills 29k stars, 8.3M npm downloads a week

Some Skills Give Context Back

Everything so far spends context. These two earn it back.

Caveman

Same prompt, same model, full transcripts. One skill loaded on the right.

Before (no skill)

> what is React

React is a JavaScript library for building user interfaces, created and maintained by Meta. It's the most widely used tool for building web frontends.

Core idea

You describe what the UI should look like for a given state, and React figures out how to update the DOM to match. You don't manipulate the DOM directly.

state change -> React re-renders components -> diffs virtual DOM -> patches real DOM

Key concepts

Components - reusable functions that return markup (JSX):

```
function Greeting({ name }) {  
  return <h1>Hello, {name}</h1>;  
}
```

Props - read-only data passed from parent to child (name above).

State - data a component owns and can change. Changing it triggers a re-render:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

Hooks - functions starting with use that plug into React features: useState (state), useEffect (side effects like

data fetching), useContext (shared data), useMemo/useCallback (caching).

Composition - you build complex UIs by nesting components:

```
App  
|-- Header  
|   |-- NavLinks  
|-- ProductList  
|   |-- ProductCard  
|   |-- ProductCard  
|   |-- ProductCard  
|-- Footer
```

Why people use it

- Declarative - less manual DOM wiring, fewer bugs
- Component reuse across a codebase
- Huge ecosystem (Next.js, React Router, React Query, etc.)
- React Native reuses the same model for iOS/Android

Things to know

- It's a library, not a framework - routing, data fetching, and build tooling come from elsewhere. Most people start with Next.js or Vite.
- Modern React is function components + hooks. Class components are legacy - you'll see them in older code but shouldn't write new ones.
- Official docs are at react.dev, and they're genuinely good.

Want me to go deeper on any part - hooks, rendering behavior, or setting up a project?

After (caveman skill)

> what is React

React = JS lib for building UI. Meta, 2013.

Core ideas:

- Components - fn returns UI. Compose tree.
- Declarative - describe state -> UI. No manual DOM poke.
- Virtual DOM - diff old/new tree, patch only changed nodes.
- One-way data flow - props down, events up.
- Hooks - useState, useEffect etc. State + side effects in fn components.

```
function Counter() {  
  const [n, setN] = useState(0);  
  return <button onClick={() => setN(n + 1)}>{n}</button>;  
}
```

Not framework - no routing/HTTP built in. Add Next.js, React Router, TanStack Query.

Render targets: web (react-dom), native (React Native).

1,858 → 596 characters. 296 → 99 words. 52 → 17 lines.

My own run, one prompt, one pair, not a benchmark. The skill advertises ~75%; this measured 68%.

RTK

A second skill, on a different axis: filtering tool output before the model reads it.

- A proxy in front of the shell. A hook rewrites git status into rtk git status.
 - **9,150 commands:** 32.5M of 36.4M tokens filtered away. 89.4%.
 - **Noisiest win most:** ps aux 97%, du 95%, git diff 94%.
 - **Terse win least:** read 20%, grep 23%. Nothing to strip.
- It counts output filtered, not money refunded.
- Context you never spend is context you still have.

```
→ ~ rtk gain
RTK Token Savings (Global Scope)
```

```
Total commands: 9150
Input tokens: 36.4M
Output tokens: 3.9M
Tokens saved: 32.5M (89.4%)
Total exec time: 476m29s (avg 3.1s)
Efficiency meter:  89.4%
```

By Command

#	Command	Count	Saved	Avg%	Time	Impact
1.	rtk find	29	25.0M	49.4%	1.4s	
2.	rtk read	353	5.3M	20.1%	0ms	
3.	rtk:toml ps aux	46	1.3M	97.0%	91ms	
4.	rtk grep	527	641.6K	23.1%	49.8s	
5.	rtk git diff agentic_...	1	37.8K	94.4%	20ms	
6.	rtk:toml ps auxww	1	35.4K	97.6%	61ms	
7.	rtk:toml ps aux -m	1	21.1K	96.0%	45ms	
8.	rtk diff	11	19.8K	57.9%	0ms	
9.	rtk:toml du -sh /User...	1	14.6K	94.7%	3.1s	
10.	rtk:toml du -shx /Use...	1	14.2K	94.6%	2.5s	

Hooks and Slash Commands

```
// .claude/settings.json
"hooks": {
  "PostToolUse": [{ "matcher": "Edit|Write",
    "hooks": [{ "type": "command", "command": "ruff format $FILE" }] }]
}
```

- Determinism for the parts that must not depend on the model.
- A hook always runs. A prompt only usually works.
- Format, lint, and test on every edit. Block edits to protected paths outright.
- Slash commands are markdown prompts; \$ARGUMENTS makes them parameterised.
- Ask the model for judgement. Ask a hook for guarantees.

Four Kinds of Loop

- The Claude Code team defines a loop as an agent repeating cycles of work until a stop condition is met.
 - **Turn-based**: you prompt, Claude stops when it judges the task done. Short work.
 - **Goal-based (/goal)**: stops when your criteria are met, or at a turn cap you set.
 - **Time-based (/loop, /schedule)**: fires on an interval. Recurring work, or watching an external system like CI.
 - **Proactive**: an event or schedule triggers it, with no human in real time.
- Start with the simplest one. Not every task needs a loop.

Loops and Skills Are the Same Argument

- A loop is only as good as its stop condition.
- Encode your manual verification steps as a SKILL.md, so Claude can check its own work end to end.
- The more quantitative the check, the easier it is for the agent to self-verify.
 - **Good:** tests passed, score above threshold, build exits zero.
 - **Weak:** “looks right”, “seems complete”.
- Give it tools that let it see the result: a browser, a test runner, a linter.

Building a Loop That Holds

- A loop's output is only as good as the system around it.
 - **Keep the codebase clean:** Claude copies the patterns already in it.
 - **Give it a way to verify:** encode what good looks like as a skill.
 - **Keep docs reachable:** framework and library docs carry current best practices.
 - **Review with a second agent:** fresh context is less biased than the one that wrote it.
- When a result misses the bar, do not just fix that result. Fix the system that produced it, so every future iteration improves.

In practice: build with Claude, then review with Codex. The bundled `/code-review` skill gives you a fresh-context reviewer for free. The Codex plugin, or `claude mcp add codex -- codex mcp-server`, gives you one that shares none of Claude's reasoning at all. A different model family, reading the diff cold.

One model agreeing with itself is not evidence.

Keeping a Loop Affordable

- Loops need boundaries, or they quietly spend everything.
 - **Right primitive, right model:** small tasks do not need agents, loops, or the big model.
 - **Clear stop criteria:** specific enough to finish, strict enough not to finish early.
 - **Pilot first:** run a slice before turning it loose on the whole job.
 - **Scripts for deterministic work:** running a script is cheaper than reasoning it out again every time.
 - **Match the interval:** do not poll faster than the thing you are watching changes.
- /usage breaks recent spend down by skill, subagent, and MCP server.

How Anthropic Ships With It

- Their own published numbers, not a vendor slogan.
 - 1M lines, under two weeks: Bun's Zig-to-Rust port. All tests green before merge.
 - 165k lines, one weekend: hundreds of agents, eight phase gates.
 - \$165k of tokens: 5.9B in, 690M out.
- “You don't fix the code. You fix the process (loop) that produced the code.”

Meanwhile, On Your Plan

Claude: Just say “Go” and I’ll generate the entire code

Me: Go!

Claude: You’ve reached your free limit



Their bill was \$165,000 of tokens. Yours is a usage limit.

One More Thing: Your Statusline

- If you live in the CLI, put the numbers where you cannot avoid seeing them.

```
Opus 5 [CAVEMAN:ULTRA] | a1moayyad | 0/1m (0%) | effort: high | 5h 36% @13:20 | 7d 37% @Fri Aug 21, 03:00 | rtk 90% (34.5m saved) | v2.1.233  
▶ auto mode on (shift+tab to cycle) · ← 2 agents
```

- **Model and mode:** which model, and which skill is currently active.
- **Context and effort:** how full the window is, and how hard it is working.
- **5h and 7d limits:** percentage used, and when each window resets.
- **rtk savings:** tokens filtered away so far.

Cost and limits stop being a surprise when they are on screen every turn. This is the cheapest habit in the whole deck.

github.com/lncch/almoayyad-claude-statusline README has setup instructions. Star the repo if you like it <3

References

- [1] Becker, Rush, Barnes, Rein (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR. arxiv.org/abs/2507.09089
- [2] DORA / Google Cloud (2025). State of AI-assisted Software Development. dora.dev/insights/balancing-ai-tensions
- [3] GitClear (2026). The Maintainability Gap: AI Code Quality Research. gitclear.com/the_ai_code_quality_maintainability_gap
- [4] Merrill, Shaw, Carlini, et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arxiv.org/abs/2601.11868
- [5] Huang, Huang, Tran, et al. (2026). Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning. arxiv.org/abs/2603.22489
- [6] Huang, Huang, Milani Fard, et al. (2026). Are AI-assisted Development Tools Immune to Prompt Injection? arxiv.org/abs/2603.21642
- [7] Adversa AI (2026, Aug 4). Nine AI coding agent incidents that deleted production data. adversa.ai/blog/ai-coding-agent-incidents
- [8] Docker (2026). Coding Agent Horror Stories: The rm -rf ~/ Incident. docker.com/blog/coding-agent-horror-stories-the-rm-rf-incident
- [9] Anthropic. Claude Code documentation. code.claude.com/docs
- [10] Anthropic. Claude Code: run prompts on a schedule (/loop, loop.md, routines). code.claude.com/docs/en/scheduled-tasks
- [11] Anthropic. Agent Skills documentation. code.claude.com/docs/en/skills
- [12] OpenAI. Codex CLI documentation. developers.openai.com/codex
- [13] Model Context Protocol. Specification 2026-07-28. modelcontextprotocol.io
- [14] AGENTS.md, an open format for agent instruction files. agents.md
- [15] Oliveira, D. (2026, Jul 6). Getting started with loops. Anthropic. claude.com/blog/getting-started-with-loops
- [16] Segner, M. (2026, Jun 18). Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents. Anthropic. claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more
- [17] Anthropic (2026, Jul 16). How Anthropic runs large-scale code migrations with Claude Code. claude.com/blog/ai-code-migration
- [18] Anthropic (2026, Jul 8). Model and effort in Claude Code: knowing more vs. trying harder. Via @ClaudeDevs, x.com/ClaudeDevs
- [19] Tools shown: github.com/obra/superpowers, github.com/yusufkaraaslan/Skill_Seekers, github.com/vercel-labs/skills and skills.sh
- [20] Token-saving tools: github.com/JuliusBrussee/caveman, github.com/rtk-ai/rtk
- [21] Status line: github.com/Incch/almoayyad-claude-statusline, forked from github.com/daniel3303/ClaudeCodeStatusLine (MIT)
- [22] OpenCode, open-source agentic CLI. opencode.ai
- [23] Community orchestrators: github.com/razzant/claudexor, github.com/kingbootoshi/codex-orchestrator. Unaffiliated with Anthropic or OpenAI.
- [24] Own measurement, 11 Aug 2026: one paired run, identical prompt and model.

Thanks for Attending!

Prepared by: Almoayyad Abuljdail